

Government Scheme Finder

1. Project Overview

Government Scheme Finder is a Flask-based web application designed to help users discover government schemes that may be relevant to their personal profile. The application allows a user to either fill out a structured profile form or describe themselves in natural language. The backend processes the submitted information, extracts a defined set of eligibility-related attributes, converts those attributes into MyScheme-compatible filters, queries the MyScheme search API, processes the returned scheme information, and renders the results in a web interface.

The current implementation is best described as an API-driven government-scheme discovery application. The provided code shows direct integration with the MyScheme search API rather than relying solely on static scheme data stored inside the project.

2. Problem Statement

India has a large number of government welfare schemes and benefits aimed at different groups of citizens. Eligibility can depend on attributes such as age, gender, state, residence, social category, disability status, and economic or demographic circumstances. For an individual user, manually identifying the schemes that may apply to them can be difficult and time-consuming.

The project addresses this problem by providing a single interface where users can describe their profile and receive a collection of potentially relevant schemes. Instead of requiring the user to manually search through schemes and determine which filters correspond to their profile, the application translates recognizable profile information into filters and uses the MyScheme search service to retrieve matching results.

3. Project Objective

Provide a simple web interface for discovering government schemes.

Support both structured form-based input and natural-language profile descriptions.

Extract recognizable user attributes from submitted text.

Convert extracted attributes into MyScheme-compatible search filters.

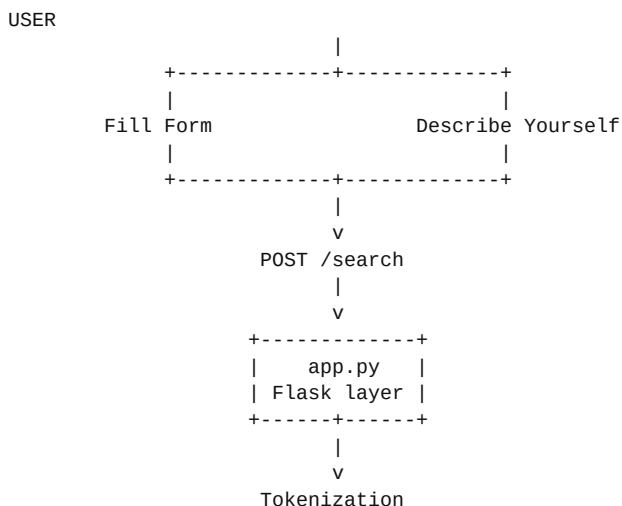
Retrieve matching schemes from the MyScheme search API.

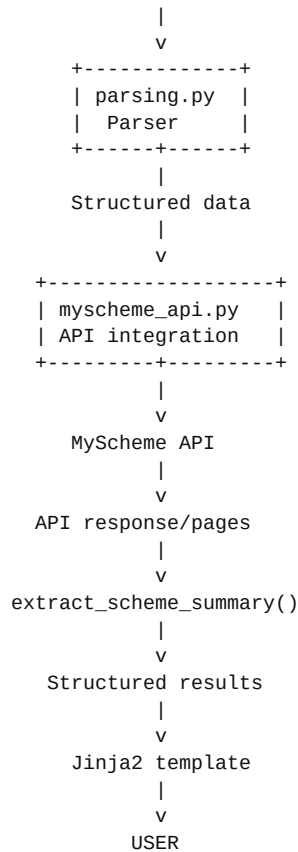
Handle multiple API result pages instead of stopping after the first page.

Normalize the API response into a structure expected by the frontend.

Display scheme names, descriptions, categories, tags and links to MyScheme details pages.

4. High-Level Architecture





5. End-to-End Data Flow

The user opens the application and receives index.html through the Flask home route.

The user either fills the structured form or switches to the natural-language input mode.

In form mode, JavaScript collects non-empty form fields and combines them into a single query string stored in a hidden input named query.

In text mode, the user's description is submitted directly through the query field.

The browser sends a POST request to /search.

app.py retrieves and trims the submitted query.

The query is tokenized using the regular expression r'\w+'.

The resulting word list is passed to process_words() in parsing.py.

parsing.py extracts recognizable attributes such as gender, age, residence, state, category, BPL, disability and senior-citizen status.

The resulting dictionary is passed to fetch_matching_schemes() in myscheme_api.py.

myscheme_api.py converts the application-level values into MyScheme filter identifiers and values.

The filters are JSON-encoded and sent to the MyScheme search API using an HTTP GET request.

The API is queried page by page until no more results remain or the reported total is reached.

extract_scheme_summary() reduces each API item to the fields required by the frontend.

app.py passes the resulting list to index.html as the Jinja variable result.

Jinja2 renders the scheme cards and displays them to the user.

6. app.py — Flask Application / Controller Layer

app.py is the central orchestration layer of the application. It initializes Flask, defines the web routes, receives user input, invokes the parser, invokes the MyScheme API layer, and sends the resulting schemes back to the frontend.

6.1 Imports

```
from flask import Flask, render_template, request
import re
from parsing import process_words
from myscheme_api import fetch_matching_schemes
```

The imports show the separation of responsibilities: Flask handles the web layer, re handles tokenization, parsing.py handles requirement extraction, and myscheme_api.py handles external scheme retrieval.

6.2 Home Route

```
@app.route('/')
def home():
    return render_template('index.html')
```

A GET request to / renders the main interface. No scheme search occurs at this stage.

6.3 Search Route

```
@app.route('/search', methods=['POST'])
```

The /search route is the main processing endpoint. It accepts POST requests generated by both the structured form and the natural-language form.

6.4 Input Handling and Tokenization

```
user_input = request.form.get('query', "").strip()
words_list = re.findall(r'\w+', user_input)
```

The backend reads the query field, removes surrounding whitespace, and tokenizes the text. If the query is empty, the page is rendered again without starting a search.

6.5 Delegation to Other Modules

```
user_data = process_words(words_list)
schemes = fetch_matching_schemes(user_data)
```

app.py does not contain the keyword rules or API filter construction itself. It delegates those responsibilities to the appropriate modules, keeping the Flask route relatively small.

7. parsing.py — User Requirement Parser

parsing.py converts a tokenized user description into a structured dictionary. The parser is keyword-based: it checks the submitted words against predefined keyword sets and a list of Indian states.

7.1 Supported Parsed Attributes

Attribute	Detection	Output
gender	male/man/boy; female/woman/girl; transgender/other	Male/Female/Other
age	First token that is entirely numeric	Integer
area_of_residence	urban/city/town or rural/village/gaon	Urban/Rural
state	Substring match against supported state list	Title-cased state
category	general/obc/sc/st/ews	Uppercase category
bpl	bpl	Yes

Attribute	Detection	Output
differently_abled	disabled/handicapped/differently-abled/ differently	Yes
senior_citizen	senior OR age >= 60	Yes

7.2 Example Parsing

Input:
65 year old male from Delhi, OBC, urban, BPL

Approximate parsed result:

```
{
  "gender": "Male",
  "age": 65,
  "area_of_residence": "Urban",
  "state": "Delhi",
  "category": "OBC",
  "bpl": "Yes",
  "differently_abled": None,
  "senior_citizen": "Yes"
}
```

7.3 Senior-Citizen Rule

The parser marks senior_citizen as Yes in either of two cases: the word senior is detected, or a numeric age has been extracted and that age is at least 60.

7.4 Important Parser Limitation

The parser is based on explicit keywords and simple numeric extraction. It is therefore not a general semantic natural-language understanding engine. A phrase that does not contain one of the recognized keywords may not be converted into the intended attribute.

8. myscheme_api.py — MyScheme Integration Layer

myscheme_api.py is responsible for translating parsed profile information into MyScheme search filters, making HTTP requests to the MyScheme API, handling pagination, and converting the API response into a frontend-compatible structure.

8.1 API Endpoint

```
MYScheme_API_URL = "https://api.myscheme.gov.in/search/v6/schemes"
```

The implementation uses the MyScheme schemes search endpoint defined in the source code.

8.2 API Key Configuration

```
MYScheme_API_KEY = os.getenv("MYScheme_API_KEY", "<fallback value in current source>")
```

The code supports reading MYSCHEME_API_KEY from an environment variable. A real credential should not be committed to source control. The current source contains a fallback credential, so that credential should be removed from the repository and rotated if it is active.

8.3 Category Mapping

```
GENERAL -> General
OBC      -> Other Backward Class (OBC)
SC       -> Scheduled Caste (SC)
ST       -> Scheduled Tribe (ST)
EWS      -> Economically Weaker Section (EWS)
```

This mapping is necessary because the application's simplified category values are converted into the longer values expected by the API integration.

8.4 Gender Mapping

```
Male -> Male
```

```
Female -> Female
Other -> Transgender
```

8.5 Filter Construction

```
build_query_filters(user_data)
```

Potential filters currently constructed:

```
gender
caste
residence
beneficiaryState
disability
age-st
```

Age is represented as a minimum and maximum equal to the supplied age. This produces an age range containing that exact value in the generated filter.

8.6 API Request Parameters

```
lang      = en
q          = JSON encoded filter list
keyword   = empty string
sort      = empty string
from      = pagination offset
size      = page size (20 by default)
```

The request also sends Accept, Origin, User-Agent and x-api-key headers.

8.7 Pagination

```
from = 0, size = 20
|
v
request page
|
v
append results
|
v
from += 20
|
v
continue until current_from >= total_available
or the returned page is empty
```

The implementation accumulates results from multiple pages rather than returning only the first page.

8.8 Response Normalization

```
{
  "fields": {
    "schemeName": ...,
    "briefDescription": ...,
    "schemeCategory": ...,
    "tags": ...,
    "slug": ...
  }
}
```

The nested fields structure is intentional because the Jinja template accesses values using expressions such as `scheme.fields.schemeName`.

8.9 Error Handling

Network and HTTP request errors are caught using `requests.exceptions.RequestException`. When a request fails during pagination, the current implementation stops fetching further pages and returns schemes collected so far instead of crashing the entire search operation.

9. index.html — Frontend / Presentation Layer

index.html is the main Jinja2 template. It provides the user interface, collects profile information, submits searches, renders scheme results, and contains client-side JavaScript for mode switching, form-to-query conversion, theme switching, and result-filter UI interactions.

9.1 Main UI Sections

Header with application name and light/dark theme toggle.

Hero section explaining the purpose of the Government Scheme Finder.

Profile finder card.

Structured Fill Form mode.

Natural-language Describe Yourself mode.

Conditional results section.

Scheme cards.

Footer.

9.2 Structured Form Fields

Age

Gender

State / UT

Area of Residence

Social Category

Annual Family Income

BPL Status

Disability Status

Highest Qualification

Occupation

Currently a Student?

Farmer / Agricultural Worker?

Senior Citizen

Marital Status

9.3 Natural-Language Mode

The natural-language interface contains a textarea named query. The user can enter a profile description in one piece of text, which is posted directly to /search.

Example:

I am a 24 year old female from rural Uttar Pradesh, belonging to SC category. I am currently pursuing graduation and my family income is around ₹2 lakh per year.

9.4 Form Mode to Common Backend Query

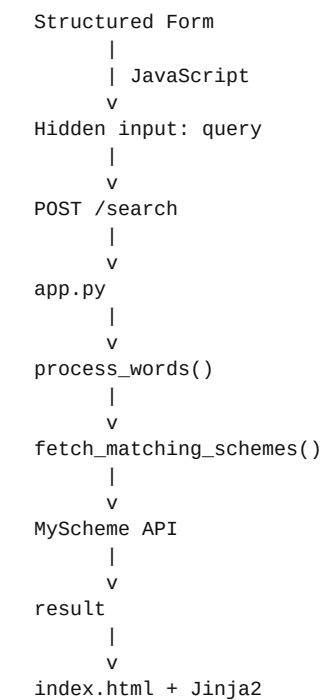
The structured form is deliberately adapted to the existing backend contract. JavaScript reads the form fields, keeps only fields with values, formats each as 'label: value', joins them with commas, and stores the result in the hidden query input. The browser then submits the same query field used by natural-language mode.

```
age: 24, gender: female, state: Uttar Pradesh,  
residence: rural, category: SC, annual income: 200000
```

9.5 Jinja2 Result Rendering

When the backend sends result=schemes to render_template(), the template checks whether result exists and then loops over the returned schemes. Each card displays the scheme category, name, brief description and available tags. If a slug exists, the card also provides a View Details link to the corresponding MyScheme scheme page.

10. Frontend-to-Backend Integration



11. Current Matching Scope vs UI Scope

A very important documentation point is that the frontend currently collects more fields than the backend parser/API filter layer currently uses. The README should distinguish these two scopes rather than claiming that every visible field affects the MyScheme query.

Current backend/API filtering or parsing	Collected by UI but not currently included in build_query_filters()
Age	Annual Family Income
Gender	Education
State	Occupation
Area of Residence	Student status
Social Category	Farmer status
Disability	Marital status
BPL is detected by parser	
Senior citizen is detected by parser	

This distinction is not a flaw in the documentation; it is an accurate description of the current implementation. These additional UI fields can be described as future integration opportunities unless other project files connect them elsewhere.

12. Technology Stack

Technology	Role	Where Used
Python	Backend language	app.py, parsing.py, myscheme_api.py
Flask	Web framework and routing	app.py
Requests	HTTP communication	myscheme_api.py
JSON	API query/response encoding	myscheme_api.py
Jinja2	Dynamic HTML rendering	index.html via Flask
HTML	Frontend structure	index.html
JavaScript	Form processing and UI behavior	index.html
CSS	Visual styling	static/style.css referenced by index.html
MyScheme API	Government scheme search source	myscheme_api.py

13. Project Structure

```

scraper/
├── static/
│   └── style.css
├── templates/
│   └── index.html
├── .gitignore
├── LICENSE
├── README.md
├── app.py
├── myscheme_api.py
└── parsing.py

```

14. Example End-to-End Scenario

User input:

"I am a 65 year old male from Delhi, OBC and I live in an urban area."

1. app.py receives the query.
2. Regex tokenization produces individual words.
3. parsing.py detects:
gender = Male
age = 65
state = Delhi
area_of_residence = Urban
category = OBC
senior_citizen = Yes
4. myscheme_api.py maps OBC to:
Other Backward Class (OBC)
5. API filters are constructed.
6. MyScheme is queried page by page.
7. Returned scheme data is normalized.
8. app.py passes the scheme list to index.html.
9. Jinja2 creates a scheme card for each result.
10. The user can open a scheme's MyScheme detail page.

15. Strengths of the Current Implementation

Clear separation between Flask routing, parsing and API integration.

Supports two user input styles while maintaining one backend search route.

Uses live API requests rather than requiring a static local scheme database for search results.

Maps application-friendly category values into API-specific values.

Supports pagination across API result pages.

Normalizes API responses for predictable frontend rendering.

Handles request failures without necessarily discarding results already retrieved.

Provides direct links to MyScheme scheme detail pages.

Provides a user-friendly profile-oriented interface.

16. Current Limitations / Areas to Improve

The parser is keyword-based and does not provide general semantic natural-language understanding.

Only the first numeric token is treated as age, which can create ambiguity in more complex descriptions.

The frontend collects several attributes that are not currently included in the API filter construction shown in `myscheme_api.py`.

BPL and senior-citizen information are parsed, but the current `build_query_filters()` function does not add BPL or senior-citizen filters.

The result filter buttons and sort selector in the frontend are currently UI interactions; the provided code does not show backend filtering or sorting based on those controls.

The current source contains a fallback API credential; credentials should be handled only through environment variables or another secure secret mechanism.

The application uses `debug=True` when executed directly, which should be disabled in production.

17. Future Enhancement Opportunities

Connect income, education, occupation, student, farmer and marital-status fields to supported API filters where available.

Add explicit API filtering for BPL and senior-citizen status if supported by the upstream API.

Improve natural-language parsing with stronger NLP or a controlled extraction layer.

Add validation and clearer user-facing error messages.

Implement actual client-side or server-side result filtering for Education, Financial, Employment and Women categories.

Implement functional result sorting instead of displaying only the current sort selector.

Add automated tests for parser rules and API filter construction.

Add structured logging rather than relying on print statements.

Move secrets completely out of source code.

Add production deployment configuration.

Consider caching carefully if permitted by the upstream service and project requirements.